



PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

IIC3413 — Implementación de Sistemas de Base de Datos — 1° 2026

LABORATORIO 1

Laboratorio: Almacenamiento de datos y heap file.
Publicación: Lunes 23 de Marzo.
Ayudantías: Viernes 20 de Marzo y Viernes 27 de Marzo.
Entrega: **Miércoles 1 de Abril 23:59 horas.**

Descripción del laboratorio

El objetivo de este primer laboratorio es conocer y familiarizarse con el almacenamiento de IIC3413DB [1], un sistema relacional que iremos desarrollando durante el curso. Para conocer el sistema de almacenamiento e identificar las componentes claves del mismo, para este laboratorio usted deberá completar la implementación del módulo `src/storage/heap_file` que nos permite manejar tuplas de una relación con una estructura de `HeapFile`.

Contexto en IIC3413DB

Buffer manager. IIC3413DB ya cuenta con un sistema de buffer manager y páginas. Esta pieza de software se encuentra implementado en los módulos `system` (archivo `buffer_manager.h`) y `storage` (archivo `page.h`).

La clase `Page` implementa el acceso “plano” a una página de datos. Cada página tiene tamaño de 4096 bytes (por lo tanto, a una página en disco la podemos visualizar cómo un arreglo de 4096 bytes). Para acceder estos datos, la clase `Page` dispone de dos métodos:

```
void read(size_t offset, size_t size, char* out)
void write(size_t offset, size_t size, char* in)
```

El método `read` copia los bytes de la página entre las posiciones `offset` y `offset + size - 1` a la variable `out`. Por otro lado, el método `write` escribe los bytes en `in` a la página entre las posiciones `offset` y `offset + size - 1` y marca la página como `dirty`, señalando que sus contenidos cambiaron con respecto a lo que hay en el disco. Adicionalmente, la clase `Page` dispone de métodos especializados para leer y escribir tipo de datos `uint8`, `int32` y `int64` cuales pueden utilizar en su implementación. Otros dos métodos importantes de la clase son `pin()` y `unpin()` los cuales hacen `pin` y `unpin` de una página.

La clase `BufferManager` implementa el buffer manager del sistema. Para solicitar una página a buffer manager, este provee el método:

```
Page& get_page(FileId file_id, uint64_t page_number)
```

que dado el identificador de un archivo (implementado por la clase `FileId`) y la dirección de una página (almacenado como un `uint64_t`), entrega la referencia a la página correspondiente (en el espacio de memoria principal ocupado por el buffer).

Es importante notar que `BufferManager` al solicitar una página con el método `get_page`, se encarga de hacer pin de la página. En otras palabras, `get_page` hace pin de la página en el buffer de manera automática. Similarmente, los `unpin()` se hacen de manera automática al destruir el objeto que usa a la página. *Por lo tanto, en su implementación no necesitan preocuparse de la secuencia de pins y unpins a una página.*

Representación de tuplas. En IIC3413DB, que es una base de datos relacional, el objeto atómico de almacenamiento es una *tupla* (también llamada *record*)¹. El módulo `relational_model` contiene la implementación de records y del esquema. Para manipular las records, IIC3413DB provee una interfaz en `record.h`. Esencialmente, un record es un vector de valores atómicos (`INT` y `STR` en el caso de IIC3413DB). Cómo pueden ver en `record.h`, el record mismo se guarda en un arreglo llamado `values` cual contiene los objetos de la clase `Value`. Para esta tarea, lo más importante de la clase `Record` es entender que podemos iterar sobre el arreglo `values`, accediendo a cada valor (`INT` o `STR`).

Los valores `INT` y `STR` se manipulan cómo objetos de la clase `Value` definida en `relational_model/value.h`. Para este laboratorio basta entender que un objeto `Value` podemos instanciar con un `int` o un `string`, y que podemos validar tipo de dato de un valor usando los métodos `is_int()` o `is_string()` de la clase `Value`.

En `record.h` pueden ver varios constructores, permitiendo instanciar un record con los valores mismos (e.g. escribiendo `Record(1,2,3)`), o con su esquema. En una tabla (relación), todos los records respetarán el mismo esquema.

El esquema para nuestros records (y tablas) está implementada en `relational_model/schema.h` mediante la clase `Schema`. Efectivamente, un objeto del tipo `Schema` es simplemente un vector dónde cada entrada contiene el nombre de la columna y su tipo de datos (en un wrapper cual llamamos `ColumnInfo` cómo pueden observar en la implementación). El nombre de cada columna es un `string`, mientras su tipo de datos (`datatype`) es un `INT` o un `STR`; quiere decir, es un dado por un `Value`².

Para manejar relaciones, IIC3413DB almacena records de una tabla (relación) a través de un heapfile.

¹Usaremos el término *tupla* y *record* para denotar lo mismo!

²En la implementación pueden ver que hay más tipos, pero no los necesitamos conocer por ahora.

HeapFile en IIC3413DB

A continuación describiremos la estructura del heapfile y sus páginas ocupadas por IIC3413DB.

Formato y manejo del HeapFile.

HeapFile (implementado en la clase `HeapFile` en el módulo `storage/heap_file`) tiene un formato muy sencillo, y consta de una secuencia de páginas identificadas por su posición en el archivo (e.g. la página 0 está en la posición 0, la página 1 en la posición 4096, etc.). Para identificar los records de una relación, nuestra clase `HeapFile` usa la estructura **RID** (record id), que se define cómo un par (**pageno**, **dir_slot**), donde **pageno** significa el número de la página, y **dir_slot** la posición del record en la página, identificado por la posición en el directorio de dicha página. A continuación explicaremos el formato de las páginas.

Formato y directorio de cada página. Cada página del HeapFile debe tener la estructura de directorio similar a los *slotted pages* explicados en las clases. Cada página (page) tiene el siguiente formato:



El header debe ser de tamaño fijo (8 bytes) y esta compuesto de dos campos: **dir_count** y **free_space**. El orden de aparición de estos campos en el header es el mismo (de izquierda a derecha – primero **dir_count** y después **free_space**). Específicamente, los campos contienen lo siguiente:

- **dir_count**: esta compuesto por 4 bytes y codifica el número de entradas en el directorio. En nuestra imagen arriba, **dir_count** = 3 y coincide con el número de records almacenados en la página, pero esto no es siempre así, porque la eliminación de un record puede causar que hay más directorios que records.
- **free_space**: esta compuesto por 4 bytes y codifica el *tamaño de espacio libre*, número de bytes libres en la página. El espacio libre se define como la cantidad de bytes sin usar entre la última entrada del directorio y el inicio de los bytes del último record (**record3** en nuestra imagen).

El directorio consta de una secuencia de punteros [**pointer**₀, ..., **pointer**_{**dir_count**-1}] donde:

- **pointer**_{*i*}: son 4 bytes que representan la posición donde se almacena el **record**_{*i*}. Esto es, la distancia desde el byte 0 en la página hasta donde empiezan los bytes del record (i.e. su **offset**). A este **pointer**_{*i*} muchas veces le vamos a decir **dir_slot** *i*.

Por ejemplo, si el directorio tiene 3 entradas, entonces el directorio tendrá largo de 12 bytes, 4 bytes por cada entrada. Por último viene la sección de la página donde se almacena el contenido de los records, que se almacenan de derecha a izquierda para dejar espacio para insertar nuevos records y entradas al directorio. Recuerden que una página contiene precisamente 4096 bytes. Por lo tanto toda la información descrita arriba debe estar almacenada en un bloque de 4096 bytes. En nuestra imagen, si la página tiene **pagenum** = 7, el **record2** tendría el **RID** = (7,1), dado que el puntero a su contenido está en la posición 1 del arreglo de directorios en esta página (los directorios contamos desde 0).

Cada **record**_{*i*} está guardado cómo la secuencia de bytes cuyos contenidos explicaremos a continuación.

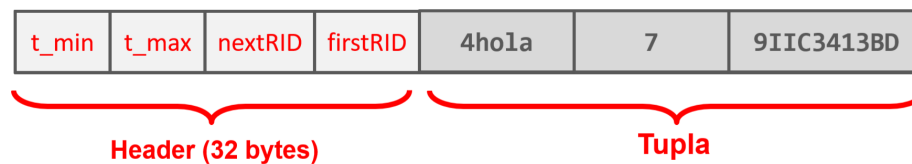
Serialización de los records. Dado que cada record necesitamos guardar no solo cómo un objeto de la clase `Record`, sino también en una página de disco, necesitamos explicar cómo se ven los bytes que se guardan para representar una tupla en una página del heap file. En otras palabras, necesitamos explicar el formato de serialización de un record. La serialización es simplemente otro nombre para la secuencia de bytes cuales describen el record al ser guardado en la página.

Recuerden que un record en IIC3413DB es simplemente un vector $t = (v_1, \dots, v_k)$ de objetos de tipo `Value`. La *serialización del contenido* de un record $t = (v_1, \dots, v_k)$ en IIC3413DB es muy sencilla y se define mediante el siguiente algoritmo (en pseudo código):

- `bytes = []` (secuencia de bytes vacía).
- Itera uno por uno sobre los valores $v_1, \dots, v_k$:
 - Si v_i es un `INT`, entonces `bytes += v_i.as_int()` – i.e. se concatena un `int64`;
 - Si v_i es un `STR`, entonces `bytes += len(v_i.as_str())+v_i.as_string()` – aquí `len(v_i.as_string())` es el largo del string guardado cómo un `uint8` quiere decir un entero de ocho bits, o un byte. Para calcular el largo de un `value` que es un string, en el código pueden escribir `value.as_string().size()`.

Para ver cómo se leen los bytes de un record desde la página, pueden revisar el método `get_record` de la clase `HeapFilePage` en el módulo `storage`.

Es importante notar que para cada record en nuestro heapfile no guardaremos solo la serialización de sus contenidos explicada arriba, sino también un header. En particular, en el disco un record se verá de la siguiente manera:



Aquí la tupla usa la serialización explicada arriba, y el header consta de los siguientes elementos:

- `t_min` – 8 bytes; timestamp de la transacción creando la tupla (por ahora ignorar);
- `t_max` – 8 bytes; timestamp de la transacción cual modificó la tupla (por ahora ignorar);
- `first_RID` – record ID de la primera versión de esta tupla; igual cómo un `RID`, consta de dos campos de 4 bytes: `page_num` y `dir_slot`. Uso de este `RID` lo explicaremos más adelante.
- `next_RID` – record ID de la siguiente versión de esta tupla (si fue modificada en algún momento); igual cómo un `RID`, consta de dos campos de 4 bytes: `page_num` y `dir_slot`. Uso de este `RID` lo explicaremos más adelante.

Todos los elementos del header se usarán para el manejo de transacciones y MVCC (multi-version concurrency control). Por ahora no necesitamos entrar a muchos detalles, pero hay que entender que cada tupla tiene una primera versión (identificada por el `first_RID`) y sus nuevas versiones forman una lista ligada implementada usando `next_RID`. Quiere decir que cada vez que una tupla se modifica, su nueva versión se escribe de nuevo al heap file y se vincula con la versión anterior actualizando su campo `next_RID`. Esta lógica la explicaremos en detalle más adelante.

En el módulo `heap_file` existen las clases `HeapFile`, cual define el funcionamiento del heapfile, y `HeapFilePage` cual define los métodos para manipular las páginas particulares del heapfile. Su trabajo en este laboratorio será completar la implementación de algunos métodos en estas clases.

Operaciones del HeapFile

Para implementar el heap file manejamos dos clases fundamentales `HeapFile`, cual maneja la lógica de acceso al heap file entero y `HeapFilePage`, cual permite trabajar con páginas particulares del heap file. Empezaremos explicando cómo uno maneja cosas desde el `HeapFile`, pero varias operaciones serán derivadas a páginas particulares.

Las operaciones fundamentales para el funcionamiento del storage es poder acceder a una tupla, insertar tuplas nuevas, o eliminar tuplas existentes.

Accediendo a tuplas. Para acceder a una tupla mediante su RID la clase `HeapFile` provee el método:

```
Record get_record(RID rid)
```

Este método consigue la tupla identificada por el RID y la devuelve cómo resultado. Si la tupla no existe, se devuelve un record sin valores.

Tuplas eliminadas. Para eliminar una tupla con `RID = (pageno, dir_slot)` en IIC3413DB, lo único que se hace es marcar el `dir_slot` con un -2. Quiere decir, *el único* cambio que se hace a la página es cambiar la entrada del directorio en la posición `dir_slot` a -2. No se cambia ningún otro byte. En particular, eliminación de una tupla no aumenta el espacio libre (sino lo pierde temporalmente), y mantiene el mismo tamaño del directorio de la página.

Inserción de un record nuevo. El tercer método crucial nos permite insertar records en una tabla. Para esto, `HeapFile` provee el siguiente método:

```
RID insert_record(Record& record, TxID tx_id, std::optional<RID> first_rid = std::nullopt)
```

Aquí el método toma un record y lo inserta en una página con el espacio suficiente para este record. En particular, esto significa que el método intenta insertar el record en la última página del heap file, revisa si hay espacio suficiente para colocar el record, y si hay, lo coloca en el espacio libre de la página y actualiza su directorio (si no hay espacio se crea una página nueva y la tupla se inserta allá). Esta inserción se llama desde la clase `HeapFile`, pero la inserción misma dentro de una página la realiza la clase `HeapFilePage`. Recuerden que el campo `free_space` del header de la página contiene la información necesaria para calcular el espacio libre y hasta dónde llega el espacio libre. Dependiendo del estado de nuestro directorio, existen dos opciones para colocar la nueva tupla en la página *cuando haya espacio suficiente para realizar la operación*:

1. Si hay alguna entrada del directorio con el valor -2 (un record eliminado), la tupla debe ser colocada en el *primer directorio* con el valor -2. Esto significa que este directorio ahora contendrá la posición del byte dónde empieza nuestro record. (Acuerden también que en el caso que el último record fue eliminado, esto no significa que su espacio pasa a ser espacio libre. Espacio libre siempre termina dónde nos señala el valor del campo `free_space`). También se actualiza el valor del campo `free_space` con la nueva información.
2. Si no hay ningún directorio con el valor -2, se crea un nuevo directorio y la tupla se ingresa al espacio libre, con el valor del nuevo directorio actualizado a la posición de nueva tupla. Es importante mencionar que en este caso el tamaño necesario para poder almacenar la tupla nueva también incluye el tamaño de un directorio nuevo (i.e. la condición es que tamaño de un directorio + tamaño de la tupla sea menor que la capacidad de `free_space`). Posteriormente se actualiza el tamaño del directorio.

En los dos casos, el método devuelve el RID de la nueva tupla. Es importante mencionar que para escribir la tupla en la página, es primero necesario serializarla cómo explicamos arriba. El largo de la tupla es el largo de su serialización, *incluyendo el tamaño del header*.

Cuando una tupla se escribe en la página, también hay que escribir su header. Los datos particulares deben ser los siguientes:

- `t_min` y `t_max` – los dos deben ser un 0; los deben escribir a la página usando el método `write_int64` de la clase `Page`.
- `first_RID` – en el caso que `first_rid` recibido en la función es igual a `std::nullopt` el valor debe ser el RID de la tupla que se está escribiendo. Quiere decir, nosotros sabemos que estamos escribiendo el record en una página, conseguimos su `pagenum` (ver la clase `Page` para ver cómo) y dado que estamos escribiendo a un `dir_pos` en esta página, el `first_rid` es igual a `(pagenum, dir_pos)`. En el caso que el valor de `first_rid` recibido es diferente a `std::nullopt` se escribe este valor.
- `next_RID` – se escribe el RID de la tupla actual – lo mismo cómo en el caso `std::nullopt` arriba.

Para escribir la tupla o su header deben usar el método `write` (o su especialización para un dato particular; e.g. `uint8`) de la clase `Page`!

Actualización de un record existente. Siguiendo método permite modificar un record. Como mencionamos anteriormente, esto no significa cambiar bytes del record actual, sino crear una versión completamente nueva de este record, colocarlo en el heap file en alguna parte con espacio, y encadenarlo con las versiones anteriores de este record. El método de la clase `HeapFile` que ocupamos para esto es:

```
RID HeapFile::update(RID rid, const Record& record, TxID tx_id)
```

Este método hace lo siguiente:

- Valida que no estamos actualizando un record eliminado; quiere decir, valida si para nuestro `RID = (pagenum, dir)` se cumple que `dir` no es -2.
- Dado que nuestro record puede ser en la mitad de una cadena de versiones del record original, se consigue la última versión. Esto pueden hacer ocupando `last = get_last_version(rid)`.
- Conseguimos el RID de la primera versión de nuestra tupla. Para esto necesitamos el header de nuestra tupla, cual podemos recuperar usando `first = get_record_header(rid).first`.
- Insertamos la nueva versión de la tupla al heap file usando `insert_record`, pero ahora para el último parámetro usamos `first`. Este insert nos devolverá el `new_rid`; el RID del record nuevo.
- Finalmente, necesitamos actualizar el header de la última versión de nuestra tupla; quiere decir el header de `last`. Para esto necesitamos conseguir la página dónde se encuentra esta tupla (`last` es su RID) y usando el método `update_record_header` de la clase `HeapFilePage` actualizar el `next`. (para entender cómo instanciar una página ver hints de la tarea; o usen su IA favorito)

Vacuum. La última operación importante del `HeapFile` es:

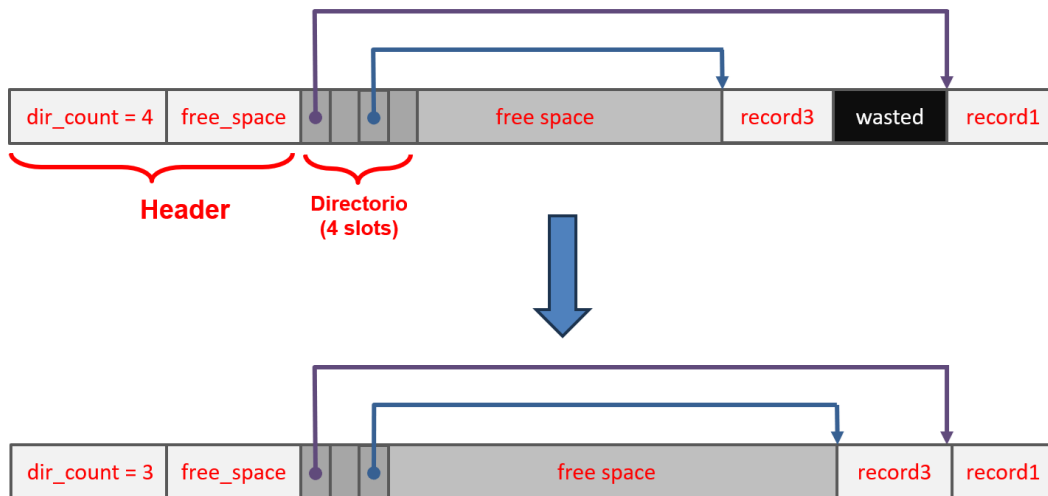
```
void vacuum()
```

En este método `HeapFile` itera página por página, y elimina todo el espacio cual se perdió por eliminación de tuplas, y algunas entradas del directorio que ya no se usan (pero no todas – ver abajo). Para esto, el algoritmo que se usa para modificar una página es el siguiente:

- Primero se detectan las entrada activas del directorio (las entradas con un valor distinto al -2). Supongamos que el directorio tiene `k` entradas y la última entrada activa está en la posición `k'`.
- Se arma una página nueva cual contiene precisamente `k'` directorios.

- Los records activos se corren hasta el final maximizando **free_space**.
- La página actual se reemplaza con la página nueva (ver imagen abajo).

Noten que la página nueva probablemente tendrá distinto número de directorios y distinto free space. Como un ejemplo, supongan que la página actual contiene tres entradas en el directorio, correspondiendo a los records r_1 , r_2 y r_3 . Si el record r_2 fue eliminado, se perdió su espacio, y la entrada dos del directorio quedó en -1. Esto significa que el `vacuum()` debería producir una página que contiene los mismos tres directorios (para mantener RIDs de las tuplas), y almacena solo r_1 y r_3 . La página nueva también tendrá el mismo orden de las tuplas que indica el directorio de la página original. La siguiente imagen contiene un ejemplo de cómo se debe ver el proceso.



Noten que en este caso solo quitamos los directorios no usados después del último directorio en uso. Para implementar esto probablemente necesitarán usar una copia de página nueva en el código y luego escribirla a página antigua (ver hints).

Estos métodos están implementados en la clase `HeapFile`, pero cada manipulación de los datos en una página se hace con un método de la clase `HeapFilePage`. Esta clase implementa la lógica de manipulación de una página. Algunos métodos helper disponible en esta clase son:

- `Record get_record(int32_t dir_pos)` – entrega el record apuntado desde el directorio `dir_pos`;
- `int32_t get_dir_count()` – retorna el número de directorios
- `void set_dir_count(int32_t new_dir_count)` – actualiza el número de directorios a `new_dir_count`;
- `int32_t get_free_space()` – retorna el tamaño de espacio libre;
- `void set_free_space(int32_t new_free_space)` – actualiza el tamaño del espacio libre a `new_free_space`;
- `int32_t get_dir(int32_t idx)` – retorna el `pointer_idx`, quiere decir la posición dónde apunta el directorio `idx`, o un -2 si este fue eliminado;
- `void set_dir(int32_t idx, int32_t new_dir_value)` – actualiza el directorio `idx` con el valor `new_dir_value`.

Algunos métodos de las clases `HeapFile` y `HepFilePage` no fueron implementados en IIC3413DB, y es su tarea completar su implementación.

Tareas para hacer en este laboratorio

Problema 1: Inserción de tuplas [2 puntos]. En la clase `HeapFilePage` falta implementar el método:

```
bool try_insert_record(Record& record, RID* out_rid, TxID tx_id, std::optional<RID> first_rid)
```

Este método es llamado por `insert_record` de la clase `HeapFile` para insertar la tupla `record`, y debería validar si la tupla cabe en la página (devolviendo `true` en este caso), y en el caso positivo, insertarla (en un directorio existente, o uno nuevo cual se agrega al final) y en el espacio libre, actualizando la información de la página de manera adecuada. El proceso de la escritura fue explicado en detalle arriba. Recuerden que para actualizar la página deben ocupar el método `write()` de la clase `Page` o una de sus variantes cuales escriben 1, 4, o 8 bytes. Adicionalmente, pueden ocupar los métodos helper de la clase `HeapFilePage`.

Problema 2: Actualización de tuplas [2 puntos]. En la clase `HeapFile` falta implementar el método:

```
bool update(RID rid, Record& record, TxID tx_id)
```

Este método debe ocupar el algoritmo de actualización de las tuplas explicado arriba y correctamente colocar una versión nueva de una tupla en el heap file.

Problema 3: Vacuum [2 puntos]. En la clase `HeapFilePage` falta implementar el método:

```
void vacuum()
```

Este método hace el vacuum al nivel de la página cómo fue explicado arriba. Como una sugerencia, para actualizar el contenido de la página al cual se le aplica `vacuum()`, se recomienda reservar un espacio del tamaño de la página `char[4096]` y armar los bytes de la página nueva en este espacio, para posteriormente copiarlo a los bytes de la página que se está procesando. Armar la modificación in-place puede ser bastante complicado en este caso. Para trabajar con una tupla, siempre pueden instanciar el objeto `record` usando la información del `schema`.

Para este laboratorio puede asumir que todos TxID que se reciben en los métodos que necesitan implementar son iguales a 0.

En términos de su implementación, hay que notar que `HeapFilePage` ya maneja los pins y unpins para ustedes (pins se hacen en el constructor y unpins con el destructor), entonces en un flujo normal no deberían tener problemas con esto. Similarmente, ocupando `write` de la clase `Page` el valor `bool dirty` de la página se actualiza de manera correcta, asegurando que sus cambios perduren en el disco cuando su programa se termine (`flush()`) de nuestro buffer hace esto de manera automática.

Es importante que su implementación siga las instrucciones anteriores, esto es, que siga el formato de la página entregado. Para la corrección se harán varios test de inserción y lectura, y las páginas del `HeapFile` deben estar exactamente como se describen anteriormente. Durante la primera semana del laboratorio se publicarán algunos test para que puedan comprobar que el formato implementado es correcto.

Finalmente, el repositorio tiene un par de tests para probar su solución. Estos tests no son exhaustivos y si su solución lo pasa no significa que obtendrán el puntaje máximo. Testear bien su solución (o la que les entrega la IA) es su tarea!

Código

El código base está disponible en [\[1\]](#). Asegúrense que están usando la última versión del sistema! Los tests se publicarán en el mismo repositorio.

Evaluación y entrega

El día límite para la entrega de esta tarea será el miércoles 1 de Abril a las 23:59 horas. Para ello se utilizará el formulario de canvas. *Su entrega debe contener un .zip de la carpeta `/src` del código. El archivo con el código se debe subir directamente al canvas.* Por último, la evaluación será en base a varios test que su solución debe responder.

Ayudantía y preguntas

El día viernes 20 de Marzo se realizará una ayudantía donde se darán más detalles sobre IIC3413DB y el laboratorio. Habrá una ayudantía adicional el viernes 27 de Marzo. Para preguntas se pide usar el foro del curso o issues de GitHub.

Referencias

[1] IIC3413DB. <https://github.com/IIC3413/2026-1>.